

INSTITUTO FEDERAL DA PARAÍBA

CAMPUS CAMPINA GRANDE

ALUNOS: MIGUEL RYAN DANTAS DE FREITAS

DISCIPLINA: PADRÕES DE PROJETO

PROFESSOR: KATYUSCO SANTOS

RELATÓRIO: BUILDER, FACTORY METHOD E ABSTRACT FACTORY

Introdução

Este relatório apresenta uma análise comparativa dos três padrões de criação fundamentais: Builder, Factory Method e Abstract Factory, baseado no material de Marcus Brizando. Todos os três padrões compartilham o objetivo comum de separar a criação de objetos de sua utilização, proporcionando maior flexibilidade e manutenibilidade ao código.

Factory Method

Conceito

O Factory Method define uma interface para criar um objeto, mas deixa as subclasses decidirem qual classe instanciar. É útil quando precisamos criar objetos sem especificar suas classes concretas.

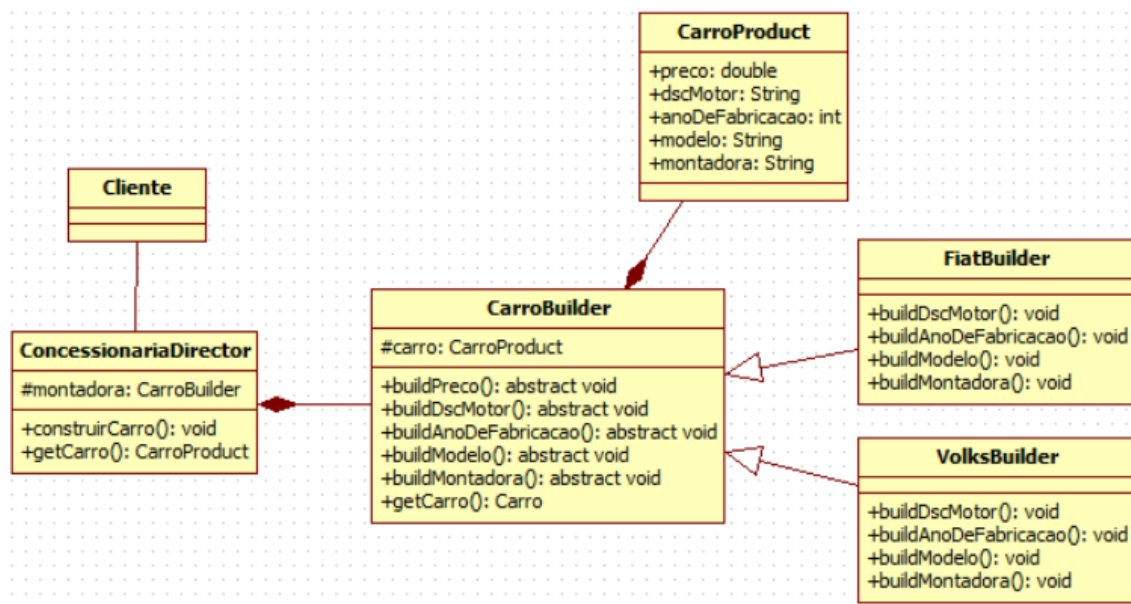
Características

- **Intenção:** Criar objetos de forma flexível, permitindo extensibilidade
- **Estrutura:** Uma interface de fábrica com implementações concretas para cada tipo de produto
- **Produto:** Objetos individuais com interface comum (ex: carros de diferentes montadoras)
- **Vantagem:** Facilidade para adicionar novos produtos sem alterar código existente
- **Desvantagem:** Criação de muitas classes pequenas (uma fábrica para cada produto)

Exemplo Prático

No material estudado, cada montadora (Fiat, Ford, Volkswagen) possui sua própria fábrica que cria um tipo específico de carro. A interface `FabricaDeCarro` define o método `criarCarro()`, e cada implementação concreta (`FabricaFiat`, `FabricaFord`) cria seu produto específico.

Diagrama UML



Abstract Factory

Conceito

O Abstract Factory fornece uma interface para criação de famílias de objetos relacionados sem especificar suas classes concretas. É uma extensão do Factory Method para trabalhar com grupos de produtos.

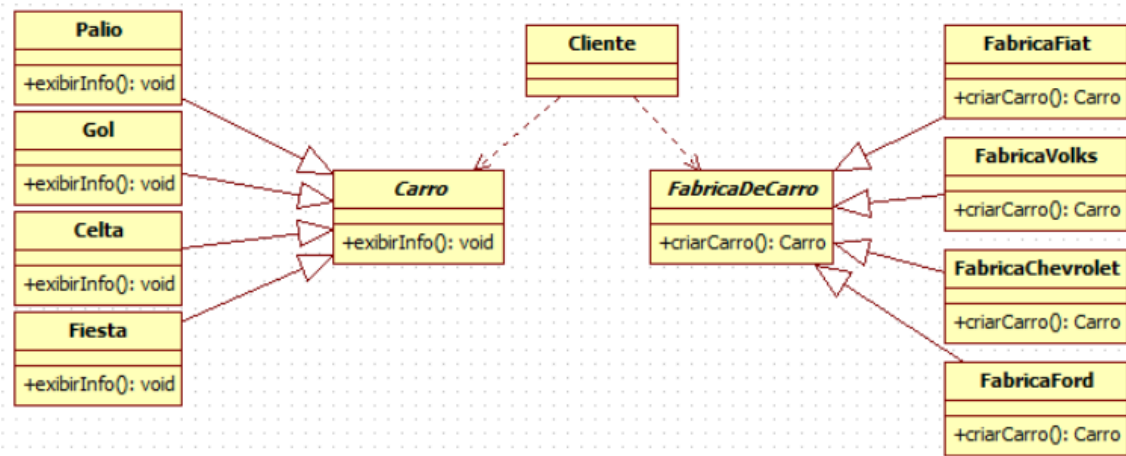
Características

- **Intenção**: Criar famílias de objetos relacionados
- **Estrutura**: Interface de fábrica com múltiplos métodos de criação para diferentes tipos de produto
- **Produto**: Múltiplas linhas de produtos (ex: carros Sedan e Populares por montadora)
- **Vantagem**: Garantia de que produtos de uma família sejam compatíveis entre si
- **Desvantagem**: Dificuldade para adicionar novos tipos de produto (requer alteração em todas as fábricas)

Exemplo Prático

No exemplo analisado, cada montadora produz duas famílias de carros: Sedan e Popular. A interface **FabricaDeCarro** define métodos `criarCarroSedan()` e `criarCarroPopular()`, e cada fábrica concreta implementa ambos os métodos para sua linha de produtos.

Diagrama UML



Builder

Conceito

O Builder separa a construção de um objeto complexo de sua representação, permitindo que o mesmo processo de construção crie diferentes representações através de passos bem definidos.

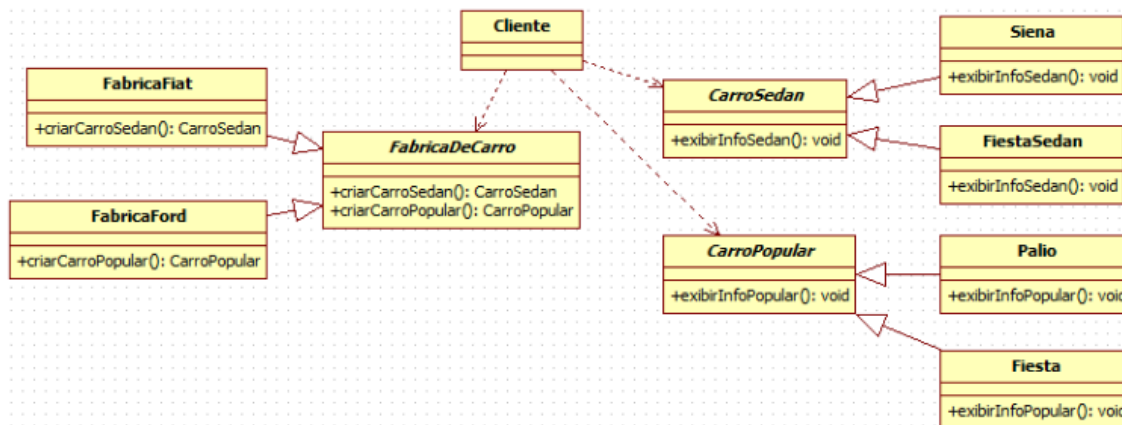
Características

- **Intenção:** Construir objetos complexos passo a passo
- **Estrutura:** Builder abstrato, Builders concretos e Director que coordena a construção
- **Produto:** Objeto único e complexo com múltiplos atributos
- **Vantagem:** Controle fino sobre o processo de construção e reutilização de algoritmos
- **Desvantagem:** Necessidade de sempre chamar métodos de construção antes de usar o produto

Exemplo Prático

O padrão constrói um **CarroProduct** através de métodos específicos (`buildPreco()`, `buildMotor()`, `buildModelo()`, etc.). O **Director** (ConcessionariaDirector) coordena a sequência de construção, enquanto os Builders concretos (FiatBuilder, VolksBuilder) definem os valores específicos para cada montadora.

Diagrama UML



Análise Comparativa

Semelhanças

- Todos encapsulam a criação de objetos
- Proporcionam flexibilidade e extensibilidade
- Reduzem o acoplamento entre código cliente e classes concretas

Diferenças Principais

Foco de Aplicação:

- **Factory Method:** Criação de objetos individuais de tipos diferentes
- **Abstract Factory:** Criação de famílias de objetos relacionados
- **Builder:** Construção passo a passo de objetos complexos

Complexidade do Produto:

- **Factory Method:** Produtos simples com interface comum
- **Abstract Factory:** Múltiplos produtos relacionados por família
- **Builder:** Produto único mas com estrutura complexa

Controle do Processo:

- **Factory Method:** Criação direta através de método único
- **Abstract Factory:** Criação de múltiplos produtos relacionados
- **Builder:** Controle detalhado sobre cada etapa da construção

Flexibilidade:

- **Factory Method:** Fácil adição de novos produtos
- **Abstract Factory:** Fácil adição de novas famílias, difícil adição de novos tipos
- **Builder:** Flexibilidade na sequência e personalização da construção

